PRODUCT REQUIREMENTS DOCUMENT (PRD) • AGENTIC SPECIFICATION

LUNGGUARD v2.0 — Backend Development & Pre-Existing Frontend Integration

Engineered for Google Antigravity IDE with Integrated GSD (Get Shit Done) Spec-Driven Workflow & Pre-Download User Consent Gates

Document Version	2.0 (Antigravity + GSD Edition)	**Development Environment**	Google Antigravity IDE (Native Agent + Terminal)
Agentic Methodology	**GSD (Get Shit Done)** Already Implemented in Workspace (`.planning/` , `STATE.md` , `ROADMAP.md`)	**Frontend Status**	**Already Present in Antigravity Workspace** (Preserve UI; wire API contract only)
Backend & AI Stack	Python 3.11 + FastAPI + PyTorch + TorchXRayVision (`densenet121-res224-all`) + OpenCV + NumPy	**Dependency Policy**	**Check-First & Ask-Before-Download** (Mandatory User Confirmation Gate)

1. PRODUCT OVERVIEW & WORKSPACE BASELINE

LUNGGUARD is an AI-assisted chest X-ray triage and reliability assessment system designed to analyze chest radiographs, compute 18 radiographic finding scores, evaluate multi-layer prediction reliability, and flag cases requiring clinical review via a unified **Trust Passport** JSON contract.

Crucial Antigravity Workspace Context:

- **Pre-Existing Frontend Files:** The frontend application is *already built and located inside the Antigravity IDE workspace*. The AI agent must inspect the existing frontend directory to discover its port, upload component, and state structure, and must **never overwrite, delete, or scaffold a new frontend from scratch**.
- **Pre-Installed GSD (Get Shit Done) Framework:** The GSD spec-driven context management system is *already implemented* in this Antigravity workspace. All backend engineering tasks must execute through GSD's externalized state files (`.planning/PROJECT.md` , `.planning/ROADMAP.md` , `.planning/STATE.md` , and atomic phase plans) to eliminate context rot across agent turns.
- **Strict Tool Inspection & Pre-Download Approval Gate:** Before installing any Python package, system binary, npm module, or downloading the pretrained `densenet121-res224-all` weights, the Antigravity agent must audit what is already installed locally and **explicitly pause to ask the user for permission** before initiating any network download.

> **MANDATORY AGENT RULE: Zero Unapproved Downloads (Ask-Before-Download Gate)**
> The Antigravity agent is **strictly forbidden** from running `pip install`, `conda install`, `apt/brew install`, `npm install`, or triggering automatic TorchXRayVision model weight downloads without prior user consent. The agent must first run non-destructive discovery commands, present a clear table of **Installed vs. Missing Tools/Packages** (with estimated download sizes), and wait for explicit user approval.

2. ANTIGRAVITY IDE & GSD (GET SHIT DONE) OPERATIONAL PROTOCOL

2.1 Leveraging the Existing GSD Setup

Because GSD is already active in Antigravity IDE, the agent must synchronize this PRD directly into the GSD control plane rather than relying on ephemeral chat memory:

- **State Synchronization (`.planning/STATE.md`):** Before starting any task, the agent reads `.planning/STATE.md` to check the active milestone, verified environment tools, and current frontend integration point. After every verified atomic task, the agent updates `STATE.md`.
- **Spec-First Execution (`.planning/ROADMAP.md`):** Map Phases 0 through 5 of this PRD into GSD phase plans (`.planning/phases/`). Use GSD verification loops (Syntax → Types → Pytest → Endpoint Smoke Test) before marking any phase complete.
- **Anti-Hallucination & File-First Discovery:** In accordance with GSD rules, the agent must inspect the existing frontend source files and Python environment using read-only file and terminal tools before proposing any code modifications.

2.2 Phase 0: Mandatory Tool Discovery & Pre-Download Consent Protocol

Before writing or running application code, the agent must execute a read-only environment audit in the Antigravity terminal and present the results to the user:

Audit Category	Read-Only Verification Command	Target Requirement	Action If Missing / Absent
Workspace & Frontend	List workspace root & inspect existing frontend config (`package.json`, `.env`, API service files)	Locate existing UI components, dev server port (e.g., 3000 or 5173), and fetch call structure	Document paths in `.planning/STATE.md`; never recreate frontend files.
GSD Control Files	Inspect `.planning/`, `.agents/`, or `.antigravity/` directories	Confirm GSD workflows and `STATE.md` / `ROADMAP.md` structure	Register LUNGGUARD backend phases inside existing GSD state files.
Python & Venv	`python3 --version` & check for existing `venv/` or `.venv/`	Python 3.11+ interpreter and isolated virtual environment	Ask user before creating a new virtual environment or installing Python.

Audit Category	Read-Only Verification Command	Target Requirement	Action If Missing / Absent
Core Python Libraries	`python3 -c "import fastapi, uvicorn, pydantic, torch, torchvision, torchxrayvision, cv2, numpy, PIL, pytest"`	Verify which required packages are already installed in the active environment	Compile exact list of missing packages and **ask user permission before running `pip install`**.
Hardware Acceleration	`python3 -c "import torch; print('CUDA:', torch.cuda.is_available(), 'MPS:', torch.backends.mps.is_available())"`	Determine whether `cuda`, `mps`, or `cpu` baseline is available	Configure device fallback automatically; never download CUDA toolkits without asking.
Model Weights Cache	Check `~/.torchxrayvision/models_data/` for `nih-pc-chex-mimic_ch-google-openi-kaggle-densenet121-d121-tw-lr001-rot45-tr15-sc15-init-best.pt`	Pretrained `densenet121-res224-all` weights (~33 MB)	If not cached locally, **ask user before triggering first weight download**.

2.3 Exact User Consent Prompt Template

When any tool, package, or model weight is missing during Phase 0, the Antigravity agent must output the following structured prompt and halt execution until the user replies:

```
[GSD ENVIRONMENT AUDIT COMPLETE — APPROVAL REQUIRED BEFORE DOWNLOAD]
1. Existing Frontend Detected: <path-to-frontend> (Port: <port>)
2. Already Installed Tools & Packages: <list of found packages>
3. Missing Packages / Weights Requiring Download:
   - <package_name==version> (Reason: <purpose>)
   - TorchXRayVision densenet121-res224-all weights (~33 MB, cached to
~/.torchxrayvision/)
May I proceed with downloading and installing the missing items listed in #3? (Yes
/ Customize)
```

2.4 Existing Frontend Adaptation Rules (Build According to Existing Frontend)

Because the frontend files are already present inside Antigravity IDE, the backend must adapt to the frontend rather than forcing a UI rewrite:

- **Step 1 — Read Frontend Source First:** Inspect the existing frontend repository (e.g., `src/`, `components/`, `services/`, or `api/`) to identify the exact URL endpoints, port numbers, form field names (e.g., `file` vs. `image`), and JSON property paths expected by the UI state.
- **Step 2 — Shape Backend Serializer to Match UI:** If the existing frontend expects specific keys inside the Trust Passport response, configure FastAPI's Pydantic response serializer and CORS origins to match the existing frontend's expectations seamlessly.
- **Step 3 — Zero Destructive Frontend Edits:** Do not replace or restyle existing frontend components; only update API base URLs or wire up the response handler where necessary.

2.5 Copy-Paste GSD Kickoff Command for Antigravity IDE

Use this exact prompt inside Antigravity IDE to launch the build under GSD rules:

```
Read the LUNGGUARD v2.0 PRD and execute Phase 0 using our existing GSD
(.planning/) workflow:
1. Inspect the existing frontend files in our workspace first so we build the
FastAPI backend strictly according to the frontend's expected routes, ports, and
JSON fields.
2. Audit all available tools, Python packages, and cached model weights in
Antigravity without installing anything yet.
3. Update .planning/STATE.md with the audit results and ASK ME FOR PERMISSION
before downloading any missing packages or TorchXRayVision weights.
```

3. SYSTEM ARCHITECTURE & END-TO-END DATA FLOW

The system architecture connects the **existing frontend in Antigravity IDE** to the modular FastAPI backend without blocking the asynchronous event loop:

```
[Pre-Existing Frontend in Antigravity Workspace]
       │  POST /api/v1/analyze (multipart/form-data: file, run_stability, run_ood)
       ▼
[FastAPI Router + In-Memory Magic-Byte Validator] ──(Reject 400 / 413 / 415 / 422
if invalid)
       │  In-Memory Bytes (io.BytesIO — zero disk writes)
       ▼
[AnalysisService Orchestrator (FastAPI run_in_threadpool Worker)]
       ├──► 1. ImageQualityGuardian (OpenCV / NumPy on raw [0, 255] image)
       │
       ├──► 2. TorchXRayVision Preprocessing Pipeline
       │       (Grayscale → xrv.datasets.normalize [-1024, 1024] → XRayCenterCrop
→ XRayResizer 224x224)
       │
       ├──► 3. Vectorized DenseNet-121 Inference & Stability Engine
(torch.inference_mode)
       │       ├── If run_stability=False: Batch (1, 1, 224, 224) Single Forward
Pass
       │       └── If run_stability=True:  Stack [Original, +5% Bright, -8%
Contrast, +Noise, +Blur]
       │                                   Batch (5, 1, 224, 224) Single
Vectorized Forward Pass
       │                                   Extract Penultimate Features (B, 1024)
+ Scores (B, 18)
       │
       ├──► 4. Reliability Hooks (Conformal Uncertainty & Penultimate Feature OOD)
       │       └── Returns measured values if calibrated, or explicit
"not_calibrated" / "not_implemented"
       │
       └──► 5. TrustGuard Deterministic Safety Engine (Priority Rules P1–P7)
               │
               ▼
[Trust Passport JSON Response] ──► [Rendered in Existing Antigravity Frontend UI]
```

4. BACKEND TECHNOLOGY STACK & VERIFICATION SPECS

Component	Technology / Package	Version Target	Role & Compatibility Notes
Programming Language	Python	`3.11.x`	Primary runtime; inspect existing Antigravity interpreter first.
API Framework	`fastapi` + `uvicorn[standard]`	`>=0.110.0`	REST endpoints, Swagger UI (`/docs`), CORS for existing frontend.
Multipart Upload	`python-multipart`	`>=0.0.9`	Required by FastAPI to parse `UploadFile` from frontend `FormData`.
Deep Learning	`torch` + `torchvision`	`>=2.2.0`	CPU baseline; auto-selects NVIDIA `cuda` or Apple Silicon `mps` if available.

Component	Technology / Package	Version Target	Role & Compatibility Notes
Medical AI Model	`torchxrayvision`	`>=1.2.0`	Provides `DenseNet(weights="densenet121-res224-all")` and X-ray transforms.
Image Processing	`opencv-python-headless` + `Pillow`	`>=4.9.0` / `>=10.2.0`	Header verification, grayscale conversion, and Laplacian blur calculation.
Numerical Engine	`numpy`	`>=1.26.0, <2.0.0`	Pinned below 2.0 to guarantee C-ABI compatibility with `torchxrayvision`.
Validation & Config	`pydantic` + `pydantic-settings`	`>=2.6.0`	Strict Trust Passport JSON contract serialization and threshold config.
Testing Suite	`pytest` + `httpx`	`>=8.0.0`	Automated GSD verification gates using synthetic in-memory X-ray fixtures.

5. DETAILED FUNCTIONAL REQUIREMENTS (FR-01 TO FR-10)

FR-01: Model Initialization & Singleton Lifespan

- Load `xrv.models.DenseNet(weights="densenet121-res224-all")` exactly **once** during FastAPI startup using the `@asynccontextmanager` lifespan hook and store the instance on `app.state.model_service`.
- Configure the model with `model.eval()` and execute all forward passes inside `with torch.inference_mode():`.
- Expose readiness via `GET /api/v1/health`. If weights have not been downloaded yet (pending user approval), the health endpoint must report `"model_loaded": false` gracefully without crashing the server.
- **Clinical Scope Limitation:** The model outputs 18 radiographic finding scores: *Atelectasis, Consolidation, Infiltration, Pneumothorax, Edema, Emphysema, Fibrosis, Effusion, Pneumonia, Pleural_Thickening, Cardiomegaly, Nodule, Mass, Hernia, Lung Lesion, Fracture, Lung Opacity,* and *Enlarged Cardiomediastinum*. It does **not** output a dedicated Tuberculosis score, and *Nodule, Mass,* and *Lung Lesion* scores must never be presented as confirmed malignancy diagnoses.

FR-02: Zero-Trust Image Upload & In-Memory Validation

- Accept `PNG`, `JPG`, and `JPEG` uploads up to **10 MB** (configurable via `MAX_UPLOAD_SIZE_MB`).
- Reject empty files (`0 bytes` → HTTP 400), oversized files (>10 MB → HTTP 413), unsupported magic headers (non-PNG `\x89PNG\r\n\x1a\n` or non-JPEG `\xff\xd8\xff` → HTTP 415), and corrupted image streams (`PIL.Image.verify()` failure → HTTP 422).
- Process all uploaded images strictly in memory via `io.BytesIO`. Never write patient radiographs to disk.

FR-03: TorchXRayVision-Compatible Image Preprocessing

1. Decode image bytes into a NumPy array and validate height and width.
2. Convert multi-channel RGB/RGBA inputs to 2D single-channel grayscale; support both 8-bit (`maxval=255`) and 16-bit (`maxval=65535`) depths.
3. Apply `xrv.datasets.normalize(img, maxval=maxval)` to map pixel intensities to the expected `[-1024.0, 1024.0]` range. **Never** apply ImageNet mean/std normalization.
4. Expand channel axis to `(1, H, W)`, apply `xrv.datasets.XRayCenterCrop()`, and apply `xrv.datasets.XRayResizer(224, engine="cv2")`.
5. Convert to a PyTorch `float32` tensor of shape `(1, 1, 224, 224)` on the active compute device.

FR-04: DenseNet-121 Inference & Feature Hook

- Execute forward pass under `torch.inference_mode()` in a worker thread (`run_in_threadpool`) so FastAPI remains non-blocking.
- Extract both the penultimate 1024-dimensional pooled feature vector (for OOD analysis) and the 18 sigmoid finding scores in a **single forward pass**.
- Return all 18 mapped scores as floats rounded to 4 decimal places alongside a sorted `top_findings` array for immediate frontend display.
- Explicitly label outputs as `uncalibrated_model_scores_not_clinical_probabilities`.

FR-05: Image Quality Guardian

Evaluates technical suitability on the standardized 8-bit grayscale representation `[0, 255]` prior to intensity shifting:

Check	Mathematical Formula / Method	Default Threshold	Warning Flag
Dimensions	$H \geq 224$ and $W \geq 224$	Min `224 × 224 px`	`LOW_RESOLUTION`
Brightness	Mean pixel intensity $\mu = (1 / HW) \sum I(i,j)$	`40.0 ≤ μ ≤ 215.0`	`UNDEREXPOSED` / `OVEREXPOSED`
RMS Contrast	Standard deviation $\sigma = \sqrt{((1 / HW) \sum (I(i,j) - \mu)^2)}$	`σ ≥ 25.0`	`LOW_CONTRAST`
Blur Estimation	Variance of Laplacian `cv2.Laplacian(img_224, cv2.CV_64F).var()`	`Var(∇²I) ≥ 40.0`	`EXCESSIVE_BLUR`
Readability / Range	Percentile span $(P_{99} - P_1) / 255.0$	`DR ≥ 0.20`	`BLANK_OR_CORRUPT_SIGNAL`

Quality Status Classification: `"acceptable"` (0 warnings), `"suboptimal"` (1 exposure/contrast/blur warning), or `"inadequate"` (≥2 warnings, `LOW_RESOLUTION`, or `BLANK_OR_CORRUPT_SIGNAL`).

FR-06: Vectorized Prediction Stability Challenge

- When `run_stability=true` (default), generate $N = 4$ controlled variations of the preprocessed `(1, 224, 224)` tensor in `[-1024, 1024]` space:
  1. **Brightness Shift (+5%):** $clip(T + 102.4, -1024, 1024)$
  2. **Contrast Change (−8%):** $clip((T - \mu_T) \times 0.92 + \mu_T, -1024, 1024)$
  3. **Mild Gaussian Noise:** $clip(T + N(0, \sigma = 25.0), -1024, 1024)$ using a fixed seed (`seed=42`) for deterministic reproducibility.
  4. **Mild Gaussian Blur:** 3×3 Gaussian kernel with $\sigma = 0.6$.
- **Single-Batch Vectorization:** Stack the original tensor and the 4 perturbations into a single `(5, 1, 224, 224)` batch tensor and execute one forward pass.
- Compute `mean_score_variation` , `max_score_delta` , `disagreement_rate` (fraction of the 18 labels flipping across the 0.50 triage reference threshold), and `stability_score` = $max(0.0, 1.0 - 5.0 \times mean_score_variation)$.
- Classify status as `"stable"` ($max_delta \le 0.12$ and $disagreement \le 0.10$), `"moderate_instability"` ($0.12 < max_delta \le 0.20$), or `"unstable"` ($max_delta > 0.20$ or $disagreement > 0.10$).

FR-07 & FR-08: Conformal Uncertainty & Out-of-Distribution (OOD) Detection

- **Conformal Prediction (FR-07):** Loads patient-independent split conformal threshold quantiles from `backend/artifacts/conformal_quantiles.json` when present to produce calibrated prediction sets and flag boundary ambiguity. When calibration data is absent in MVP, returns `{"status": "not_calibrated", "prediction_set": null, "ambiguous_findings": []}` .
- **OOD Detection (FR-08):** Uses penultimate 1024-d DenseNet features against reference distribution parameters in `backend/artifacts/ood_reference_stats.npz` . When reference stats are absent in MVP, returns `{"status": "not_implemented", "score": null, "threshold": null}` . Never fabricate placeholder numerical values.

FR-09 & FR-10: TrustGuard Safety Engine & Unified Trust Passport

TrustGuard synthesizes all active reliability outputs into a deterministic decision without ever declaring a patient "healthy" or "cleared":

Priority	Rule Condition	TrustGuard Decision	Reason Code
P1	`image_quality.status == "inadequate"`	`"unreliable_input"`	`IMAGE_QUALITY_INADEQUATE`
P2	`ood.status == "out_of_distribution"`	`"review_recommended"`	`OOD_INPUT_DETECTED`
P3	`stability.status == "unstable"`	`"review_recommended"`	`PREDICTION_INSTABILITY_HIGH`

Priority	Rule Condition	TrustGuard Decision	Reason Code
P4	`image_quality.status == "suboptimal"`	`"review_recommended"`	`IMAGE_QUALITY_SUBOPTIMAL`
P5	`uncertainty.status == "calibrated"` with ambiguous set	`"review_recommended"`	`CONFORMAL_SET_AMBIGUOUS`
P6	Any finding score ≥ *0.50* OR uncalibrated advanced modules	`"review_recommended"`	`ELEVATED_SCORE_OR_UNCALIBRATED_PIPELINE_REVIEW`
P7	All active checks pass & all scores < *0.50* (with calibrated modules)	`"standard_triage"`	`ALL_RELIABILITY_CHECKS_PASSED`

6. REST API SPECIFICATION & TRUST PASSPORT JSON CONTRACT

Endpoint	Method	Request Payload / Params	Response Summary
`/api/v1/health`	`GET`	None	`{"status": "healthy", "model_loaded": true, "model_name": "DenseNet-121", "device": "cpu"}`
`/api/v1/analyze`	`POST`	`multipart/form-data` field: `file` ; optional query params: `run_stability=true` , `run_ood=false`	Returns complete **Trust Passport JSON** (see canonical contract below) and caches result in a 100-item in-memory LRU store.
`/api/v1/analysis/{analysis_id}`	`GET`	Path param: `analysis_id` (UUID)	Retrieves cached Trust Passport from memory (or SQLite if enabled) so frontend page reloads succeed.
`/api/v1/model-info`	`GET`	None	Returns model architecture, weights version, input shape `224x224x1` , and all 18 label names.
`/docs`	`GET`	Browser navigation	Interactive FastAPI Swagger UI documentation.

Canonical Trust Passport JSON Contract (Backward & Forward Compatible)

```
{
  "analysis_id": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
  "timestamp_utc": "2026-11-04T10:15:30Z",
  "processing_time_ms": 178.4,
  "status": "completed",
  "model": {
    "name": "DenseNet-121",
    "version": "densenet121-res224-all",
    "device": "cpu",
    "score_type": "uncalibrated_model_scores"
  },
  "predictions": {
    "Atelectasis": 0.4215,
    "Cardiomegaly": 0.1804,
    "Effusion": 0.3112,
    "... (all 18 pathologies included at top level for direct UI mapping)": 0.0510
  },
  "top_findings": [
    {"label": "Atelectasis", "score": 0.4215},
    {"label": "Effusion", "score": 0.3112},
    {"label": "Cardiomegaly", "score": 0.1804}
  ],
  "image_quality": {
    "status": "acceptable",
    "brightness": 112.4,
    "contrast": 48.2,
    "blur_score": 125.6,
    "dynamic_range": 0.86,
    "dimensions": {"width": 1024, "height": 1024},
    "warnings": [],
    "review_recommended": false
  },
  "stability": {
    "status": "stable",
    "tests_run": 4,
    "score": 0.914,
    "mean_score_variation": 0.0172,
    "max_score_delta": 0.0510,
    "disagreement_rate": 0.0
  },
  "uncertainty": {
    "status": "not_calibrated",
    "prediction_set": null,
    "ambiguous_findings": []
  },
  "ood": {
    "status": "not_implemented",
    "score": null
  },
  "trustguard": {
    "decision": "review_recommended",
    "reasons": [
      "ELEVATED_SCORE_OR_UNCALIBRATED_PIPELINE_REVIEW"
```

7. WORKSPACE FOLDER STRUCTURE (WITH EXISTING FRONTEND & GSD)

```
LUNGGUARD/
├── .planning/                        # EXISTING GSD (Get Shit Done) Spec & State
Directory
│   ├── PROJECT.md                    # Global project context
│   ├── ROADMAP.md                    # Phase breakdown & verification status
│   ├── STATE.md                      # Active blackboard state updated after every
task
│   └── phases/                       # Atomic GSD execution plans (Phase 0 to Phase
5)
├── frontend/                         # EXISTING Frontend Application (DO NOT
OVERWRITE)
│   └── ...                           # Inspect existing upload & Trust Passport UI
components
├── backend/                          # Target Backend Directory to Build
│   ├── app/
│   │   ├── main.py                   # FastAPI app, lifespan model loader, CORS
middleware
│   │   ├── config.py                 # Thresholds, upload limits, allowed frontend
origins
│   │   ├── schemas.py                # Pydantic v2 TrustPassport & error contracts
│   │   ├── api/
│   │   │   └── routes.py             # /health, /analyze, /analysis/{id}, /model-
info
│   │   ├── services/
│   │   │   ├── model_service.py      # Singleton DenseNet-121 holder
│   │   │   └── analysis_service.py # Threadpool pipeline orchestrator & in-memory
cache
│   │   ├── ml_engine/
│   │   │   ├── model.py              # TorchXRayVision DenseNet-121 loader &
feature hook
│   │   │   ├── preprocessing.py      # Grayscale, xrv.normalize, XRayCenterCrop,
XRayResizer
│   │   │   ├── inference.py          # Batch inference under torch.inference_mode()
│   │   │   ├── image_quality.py      # Brightness, contrast, Laplacian blur,
dynamic range
│   │   │   ├── stability.py          # Vectorized 5-sample perturbation challenge
│   │   │   ├── uncertainty.py        # Conformal prediction module & honest
fallback
│   │   │   ├── ood_detection.py      # Penultimate feature distance OOD & honest
fallback
│   │   │   └── trustguard.py         # Deterministic safety rule evaluator
│   │   └── utils/
│   │       └── image_validation.py # Magic-byte, size, and PIL integrity checks
│   ├── tests/
│   │   ├── conftest.py               # Synthetic in-memory X-ray generator &
TestClient
│   │   ├── test_health.py
│   │   ├── test_upload.py
│   │   ├── test_inference.py
│   │   └── test_trustguard.py
│   ├── requirements.txt
│   └── README.md
```

8. GSD PHASED EXECUTION ROADMAP & ACCEPTANCE CRITERIA

GSD Phase	Priority	Core Deliverables & GSD Actions	GSD Verification Gate
Phase 0: Tool Audit & Frontend Discovery	Critical	Inspect existing frontend files; check Python/PyTorch/XRV/OpenCV availability; log findings in `.planning/STATE.md`; **ask user before any download**.	User explicitly approves download list (or confirms offline cached packages).
Phase 1: Core AI Engine	Critical	Implement `config.py`, `model.py`, `preprocessing.py`, and `inference.py` using `densenet121-res224-all`.	`pytest backend/tests/test_inference.py` passes; all 18 labels verified.
Phase 2: Reliability & TrustGuard	High	Implement `image_quality.py`, vectorized `stability.py`, `uncertainty.py`, `ood_detection.py`, and `trustguard.py`.	`pytest backend/tests/test_trustguard.py` passes deterministic P1–P7 checks.
Phase 3: FastAPI & Security	Critical	Implement `schemas.py`, `image_validation.py`, `model_service.py`, `analysis_service.py`, `routes.py`, and `main.py`.	`pytest backend/tests/test_health.py backend/tests/test_upload.py` passes.
Phase 4: Existing Frontend Integration	Critical	Configure CORS for the existing frontend port; align JSON field mapping with existing frontend components; ensure `FormData` omits manual `Content-Type`.	End-to-end upload from existing frontend displays live Trust Passport data.

Final MVP Acceptance Criteria & Medical Safety

- **Tool & Download Compliance:** Zero packages or model weights are downloaded without first running the Phase 0 local audit and receiving explicit user approval.
- **Frontend Preservation:** The existing frontend in Antigravity IDE remains intact and renders live backend responses from `POST /api/v1/analyze`.
- **GSD State Continuity:** Every completed phase is recorded in `.planning/STATE.md` and verified via automated `pytest` gates.
- **Clinical Safety Scope:** LUNGGUARD operates strictly as an experimental AI triage prototype. All finding scores are labeled as uncalibrated model activations, unavailable calibration modules report explicit statuses (`not_calibrated`, `not_implemented`), and TrustGuard never declares a patient healthy or disease-free.